

Отчёт по лабораторной работе №8

Программирование цикла. Обработка аргументов командной строки.

Мещерякова София Андреевна

Содержание

1	Цель работы	4
2	Выполнение лабораторной работы	5
2.1	Реализация циклов в NASM	5
2.2	Обработка аргументов командной строки.	9
2.3	Задание для самостоятельной работы	13
3	Выводы	16

Список иллюстраций

2.1	Создаем каталог и файл для лабораторной работы №8	5
2.2	Текст программы (Листинг 8.1)	6
2.3	Запуск программы	6
2.4	Текст измененной программы	7
2.5	Запуск программы	7
2.6	Текст измененной программы	8
2.7	Запуск программы	9
2.8	Создание нового файла	9
2.9	Текст программы (Листинг 8.2)	10
2.10	Запуск программы	10
2.11	Создание нового файла	11
2.12	Текст программы (Листинг 8.3)	11
2.13	Запуск программы	11
2.14	Текст программы для вычисления произведения значений	12
2.15	Запуск программы	12
2.16	Текст программы для вычисления суммы значений	14
2.17	Запуск программы (все верно)	14
2.18	Запуск программы (все верно)	15

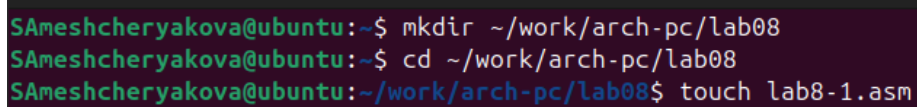
1 Цель работы

Приобретение навыков написания программ с использованием циклов и обработкой аргументов командной строки.

2 Выполнение лабораторной работы

2.1 Реализация циклов в NASM

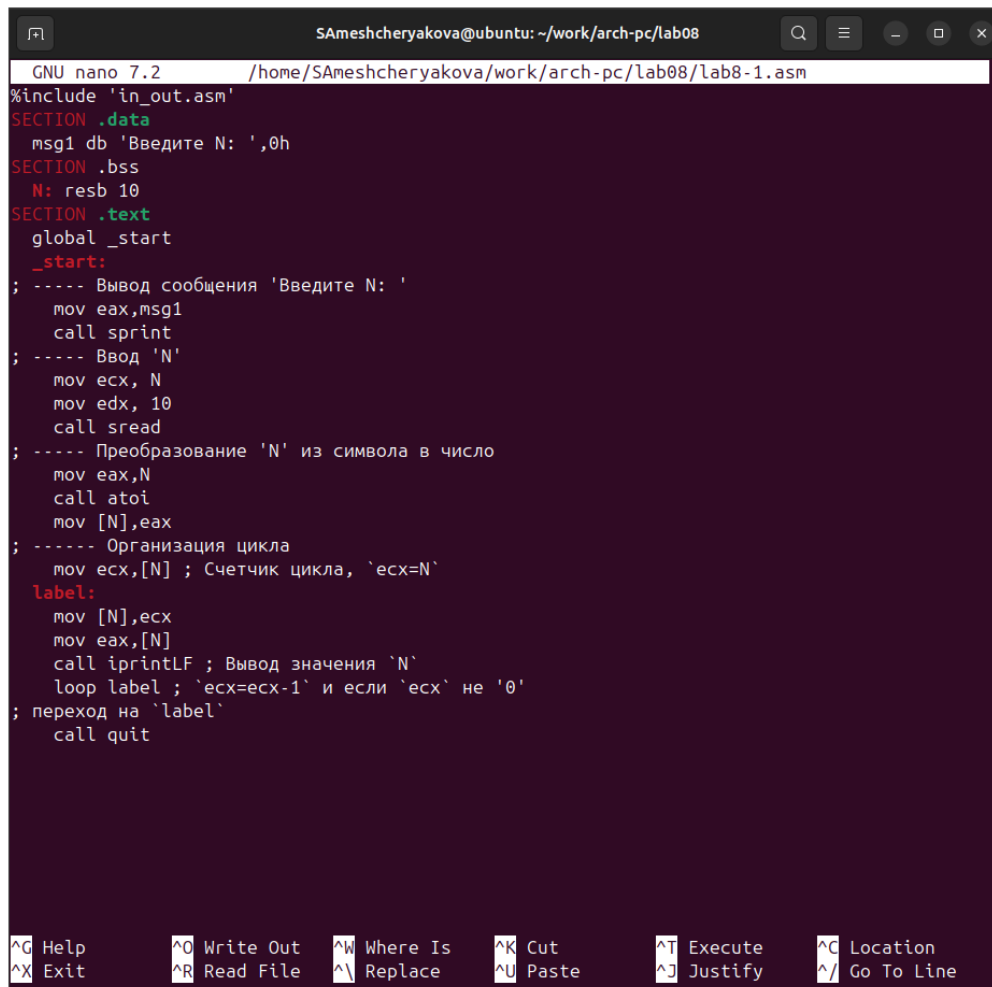
1. Создадим каталог для программ лабораторной работы № 8, перейдём в него и создадим файл lab8-1.asm (рис. 2.1).



```
SAmeshcheryakova@ubuntu:~$ mkdir ~/work/arch-pc/lab08
SAmeshcheryakova@ubuntu:~$ cd ~/work/arch-pc/lab08
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ touch lab8-1.asm
```

Рисунок 2.1: Создаем каталог и файл для лабораторной работы №8

В качестве примера рассмотрим программу, которая выводит значение регистра есх. Введем файл программу из листинга 8.1 (рис. 2.2).



```
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab08/lab8-1.asm
%include 'in_out.asm'
SECTION .data
    msg1 db 'Введите N: ',0h
SECTION .bss
    N: resb 10
SECTION .text
    global _start
    _start:
; ----- Вывод сообщения 'Введите N: '
    mov eax,msg1
    call sprint
; ----- Ввод 'N'
    mov ecx, N
    mov edx, 10
    call sread
; ----- Преобразование 'N' из символа в число
    mov eax,N
    call atoi
    mov [N],eax
; ----- Организация цикла
    mov ecx,[N] ; Счетчик цикла, `ecx=N`
label:
    mov [N],ecx
    mov eax,[N]
    call iprintLF ; Вывод значения `N`
    loop label ; `ecx=ecx-1` и если `ecx` не `0`
; переход на `label`
    call quit
```

Terminal window showing assembly code in nano editor. The code includes directives for data, bss, and text sections, and assembly instructions for printing, reading, and looping. The bottom of the window shows nano editor shortcuts.

Рисунок 2.2: Текст программы (Листинг 8.1)

Создаем исполняемый файл и запускаем его (рис. 2.3).

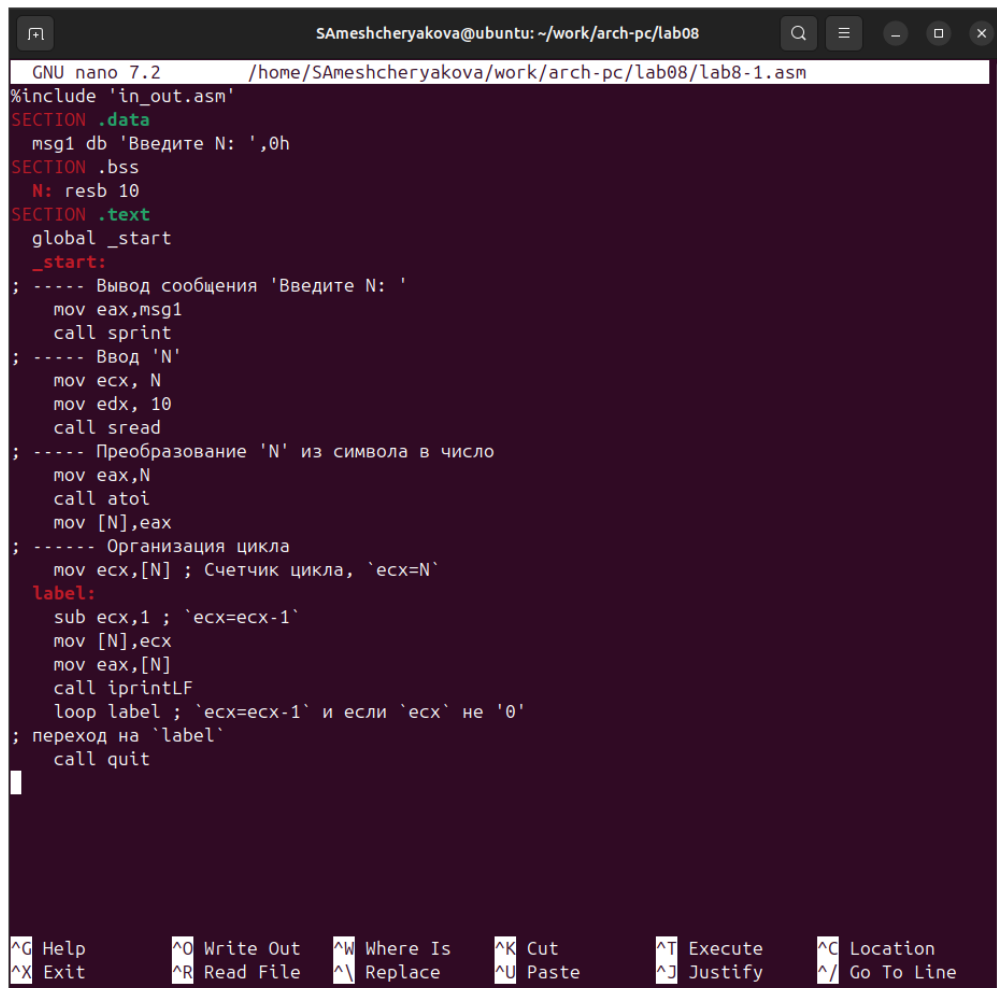


```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 10
10
9
8
7
6
5
4
3
2
1
```

Terminal window showing the compilation of lab8-1.asm to lab8-1.o using nasm, linking it to an executable lab8-1 using ld, and then running the executable. The program prompts for input 'N' and prints a countdown from 10 to 1.

Рисунок 2.3: Запуск программы

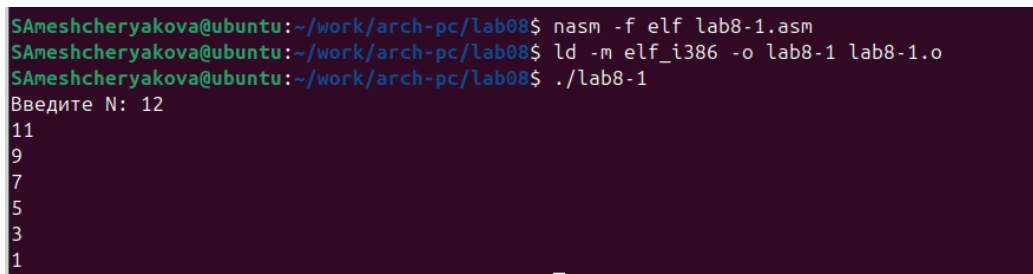
Изменяем файл, добавив изменение значения регистра в цикле (рис. 2.4).



```
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab08/lab8-1.asm
#include 'in_out.asm'
SECTION .data
    msg1 db 'Введите N: ',0h
SECTION .bss
    N: resb 10
SECTION .text
    global _start
_start:
; ----- Вывод сообщения 'Введите N: '
    mov eax,msg1
    call sprint
; ----- Ввод 'N'
    mov ecx, N
    mov edx, 10
    call sread
; ----- Преобразование 'N' из символа в число
    mov eax,N
    call atoi
    mov [N],eax
; ----- Организация цикла
    mov ecx,[N] ; Счетчик цикла, `ecx=N`
label:
    sub ecx,1 ; `ecx=ecx-1`
    mov [N],ecx
    mov eax,[N]
    call iprintLF
    loop label ; `ecx=ecx-1` и если `ecx` не '0'
; переход на `label`
    call quit
```

Рисунок 2.4: Текст измененной программы

Создаем исполняемый файл и запускаем его (рис. 2.5).



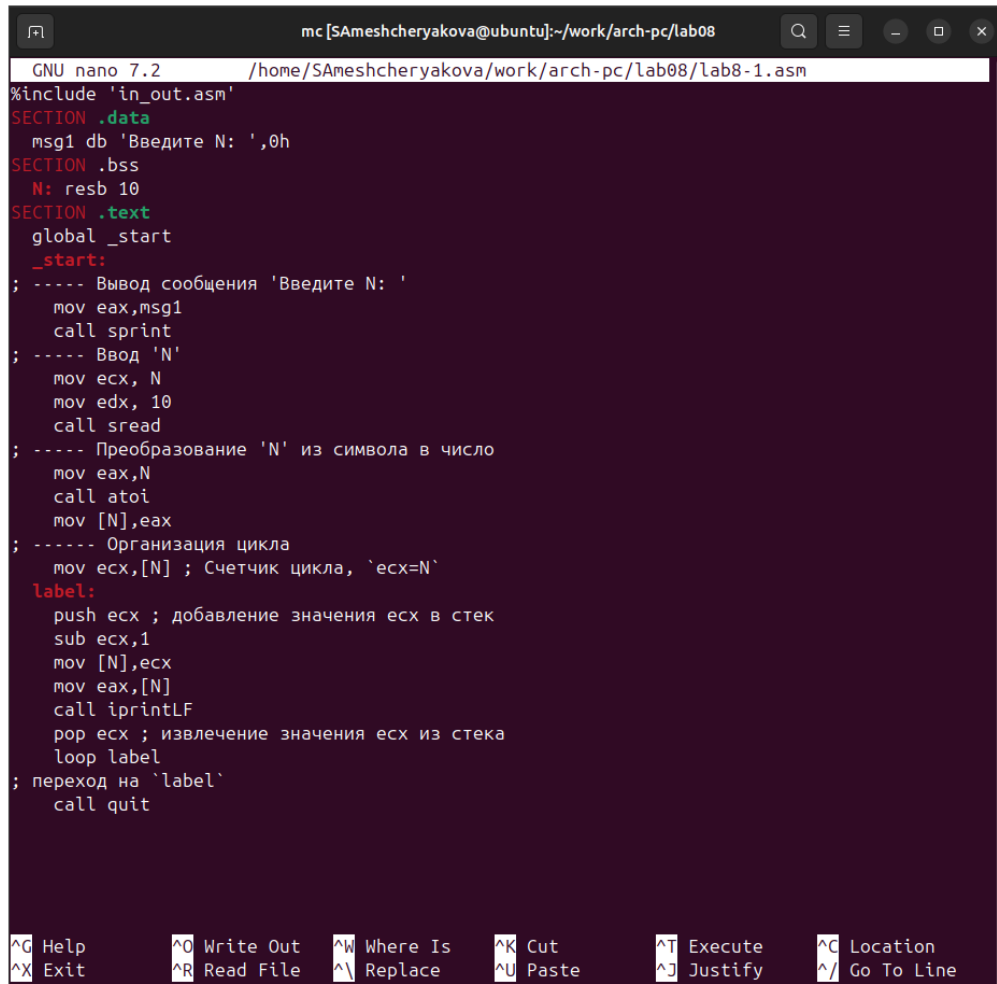
```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 12
11
9
7
5
3
1
```

Рисунок 2.5: Запуск программы

Регистр `ecx` принимает значения 11, 9, 7, 5, 3, 1 (на вход подается число 12, в цикле регистр уменьшается на 2).

Число проходов цикла не соответствует числу `N`.

Изменяем файл, чтобы он работал корректно (рис. 2.6).



```
mc [SAmeshcheryakova@ubuntu]:~/work/arch-pc/lab08
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab08/lab8-1.asm
#include 'in_out.asm'
SECTION .data
    msg1 db 'Введите N: ',0h
SECTION .bss
    N: resb 10
SECTION .text
    global _start
    _start:
; ----- Вывод сообщения 'Введите N: '
    mov eax,msg1
    call sprint
; ----- Ввод 'N'
    mov ecx, N
    mov edx, 10
    call sread
; ----- Преобразование 'N' из символа в число
    mov eax,N
    call atoi
    mov [N],eax
; ----- Организация цикла
    mov ecx,[N] ; Счетчик цикла, `ecx=N`
label:
    push ecx ; добавление значения ecx в стек
    sub ecx,1
    mov [N],ecx
    mov eax,[N]
    call iprintLF
    pop ecx ; извлечение значения ecx из стека
    loop label
; переход на `label`
    call quit
```

Рисунок 2.6: Текст измененной программы

Создаем исполняемый файл и запускаем его (рис. 2.7).


```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 13
12
11
10
9
8
7
6
5
4
3
2
1
0
```

Рисунок 2.7: Запуск программы

В данном случае число проходов цикла равна числу N.

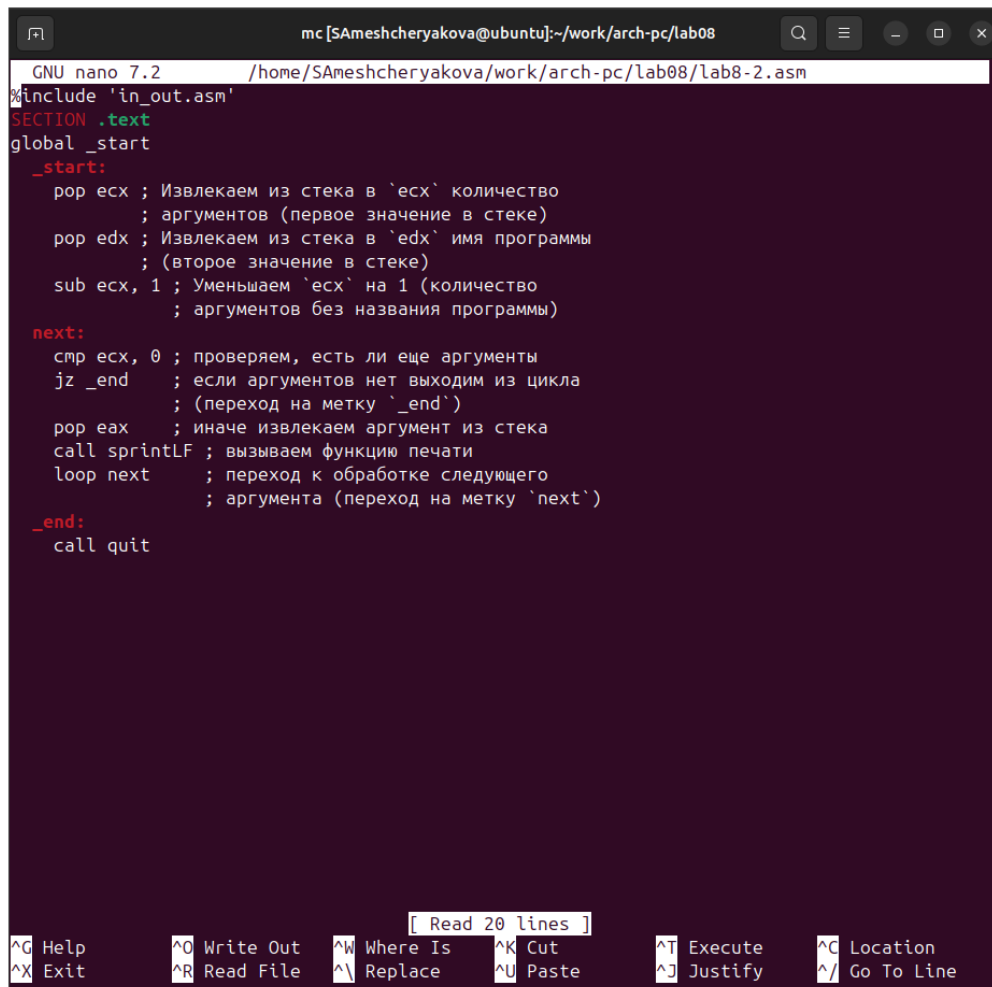
2.2 Обработка аргументов командной строки.

Создаем новый файл (рис. 2.8).

```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ touch lab8-2.asm
```

Рисунок 2.8: Создание нового файла

Введем файл программу из листинга 8.2 (рис. 2.9).



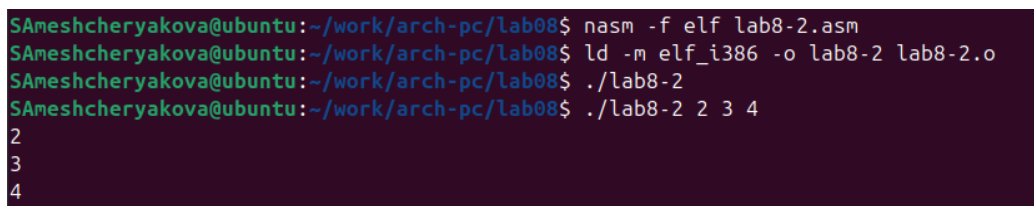
```
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab08/lab8-2.asm
#include 'in_out.asm'
SECTION .text
global _start
_start:
    pop ecx ; Извлекаем из стека в `ecx` количество
             ; аргументов (первое значение в стеке)
    pop edx ; Извлекаем из стека в `edx` имя программы
             ; (второе значение в стеке)
    sub ecx, 1 ; Уменьшаем `ecx` на 1 (количество
             ; аргументов без названия программы)

next:
    cmp ecx, 0 ; проверяем, есть ли еще аргументы
    jz _end    ; если аргументов нет выходим из цикла
             ; (переход на метку `_end`)
    pop eax    ; иначе извлекаем аргумент из стека
    call printf ; вызываем функцию печати
    loop next  ; переход к обработке следующего
             ; аргумента (переход на метку `next`)

_end:
    call quit
```

Рисунок 2.9: Текст программы (Листинг 8.2)

Создаем исполняемый файл и проверяем его работу, указав аргументы (рис. 2.10).



```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ nasm -f elf lab8-2.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-2 lab8-2.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ ./lab8-2
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ ./lab8-2 2 3 4
2
3
4
```

Рисунок 2.10: Запуск программы

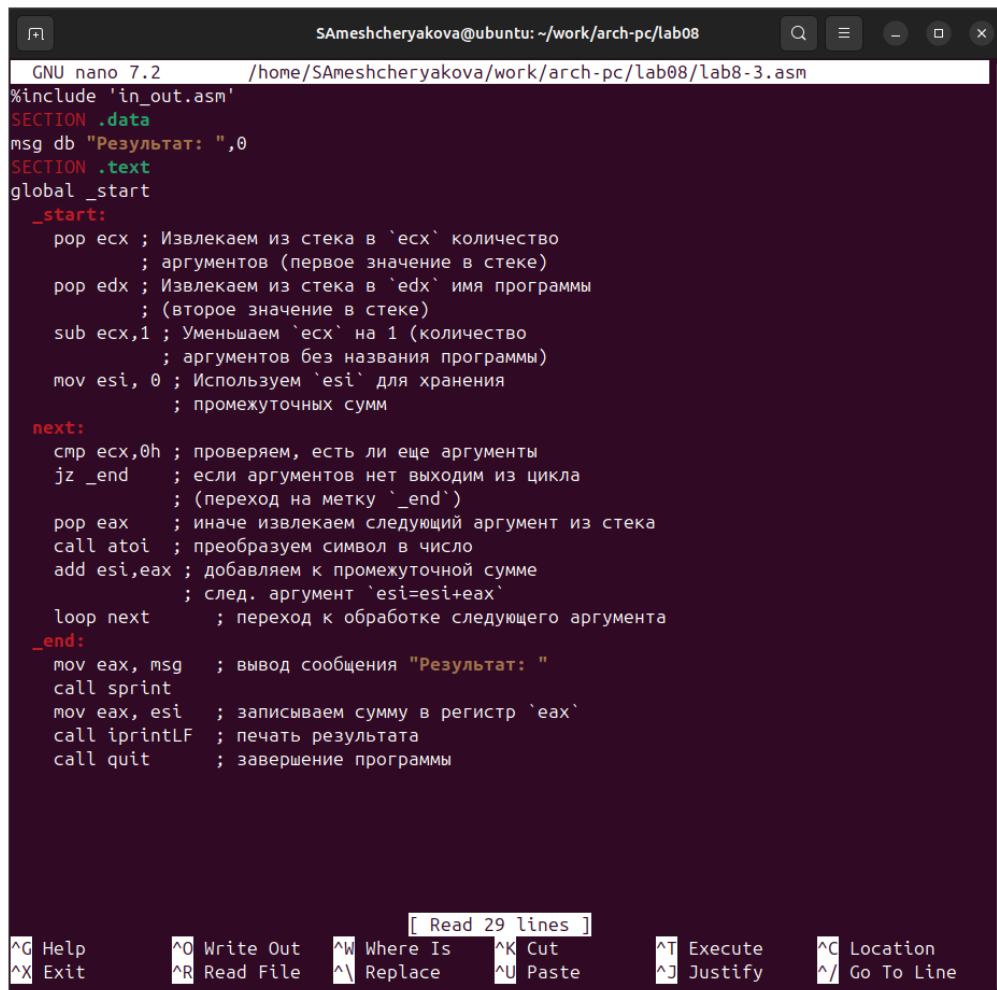
Программой было обработано 3 аргумента.

Создаем новый файл lab8-3.asm (рис. 2.11).

```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ touch lab8-3.asm
```

Рисунок 2.11: Создание нового файла

Введем файл программу из листинга 8.3 (рис. 2.12).



```
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab08/lab8-3.asm
#include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
SECTION .text
global _start
_start:
    pop ecx ; Извлекаем из стека в `ecx` количество
              ; аргументов (первое значение в стеке)
    pop edx ; Извлекаем из стека в `edx` имя программы
              ; (второе значение в стеке)
    sub ecx,1 ; Уменьшаем `ecx` на 1 (количество
              ; аргументов без названия программы)
    mov esi, 0 ; Используем `esi` для хранения
              ; промежуточных сумм
next:
    cmp ecx,0h ; проверяем, есть ли еще аргументы
    jz _end    ; если аргументов нет выходим из цикла
              ; (переход на метку `_end`)
    pop eax    ; иначе извлекаем следующий аргумент из стека
    call atoi  ; преобразуем символ в число
    add esi,eax ; добавляем к промежуточной сумме
              ; след. аргумент `esi=esi+eax`
    loop next  ; переход к обработке следующего аргумента
_end:
    mov eax,msg ; вывод сообщения "Результат: "
    call sprint
    mov eax,esi ; записываем сумму в регистр `eax`
    call iprintLF ; печать результата
    call quit   ; завершение программы
```

Рисунок 2.12: Текст программы (Листинг 8.3)

Создаём исполняемый файл и запускаем его, указав аргументы (рис. 2.13).

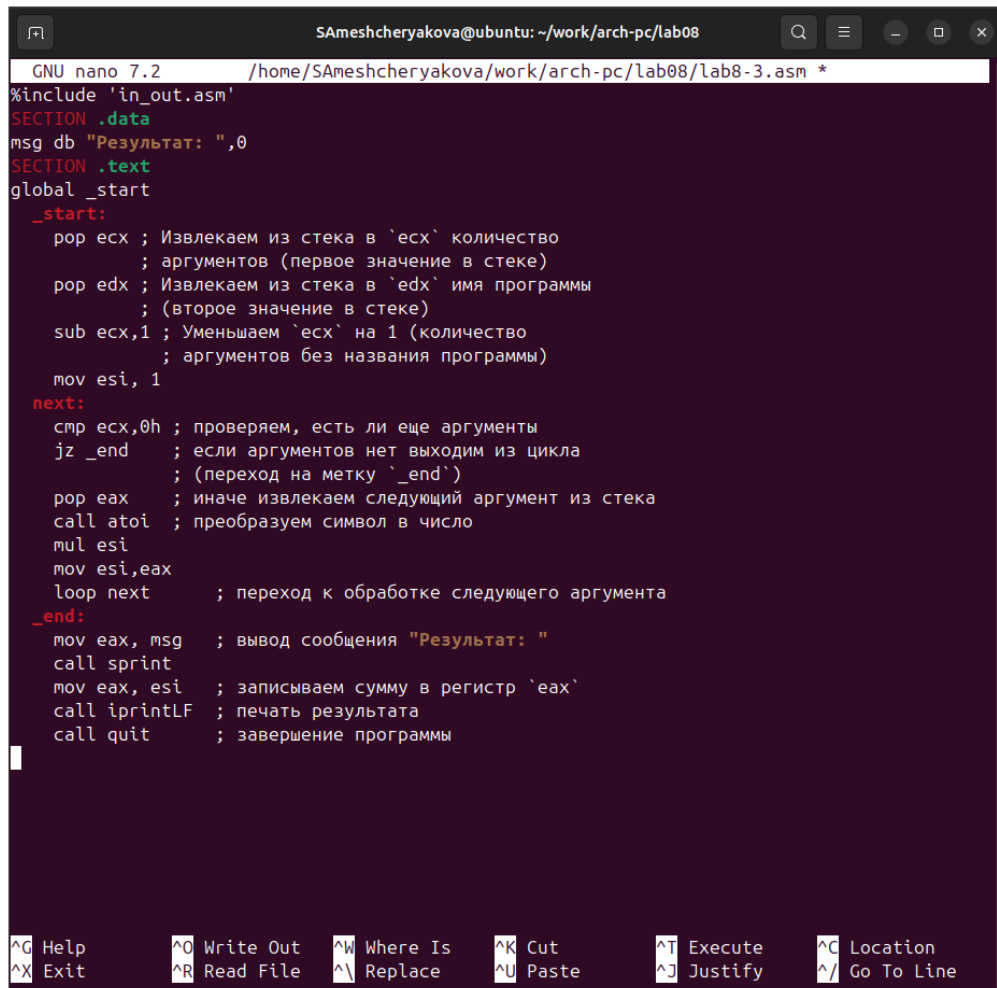
```

SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-3 lab8-3.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ ./lab8-3 12 13 7 10 5
Результат: 47

```

Рисунок 2.13: Запуск программы

Изменяем его, чтобы вычислялось произведение вводимых значений (рис. 2.14).



```

GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab08/lab8-3.asm *
%include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
SECTION .text
global _start
_start:
    pop ecx ; Извлекаем из стека в `ecx` количество
              ; аргументов (первое значение в стеке)
    pop edx ; Извлекаем из стека в `edx` имя программы
              ; (второе значение в стеке)
    sub ecx,1 ; Уменьшаем `ecx` на 1 (количество
              ; аргументов без названия программы)
    mov esi, 1
next:
    cmp ecx,0h ; проверяем, есть ли еще аргументы
    jz _end    ; если аргументов нет выходим из цикла
              ; (переход на метку `_end`)
    pop eax    ; иначе извлекаем следующий аргумент из стека
    call atoi  ; преобразуем символ в число
    mul esi
    mov esi,eax
    loop next  ; переход к обработке следующего аргумента
_end:
    mov eax, msg ; вывод сообщения "Результат: "
    call sprint
    mov eax, esi ; записываем сумму в регистр `eax`
    call iprintLF ; печать результата
    call quit    ; завершение программы

```

Рисунок 2.14: Текст программы для вычисления произведения значений

Создаём исполняемый файл и запускаем его, указав аргументы (рис. 2.15).

```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-3 lab8-3.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ ./lab8-3 1 2 3 4 5
Результат: 120
```

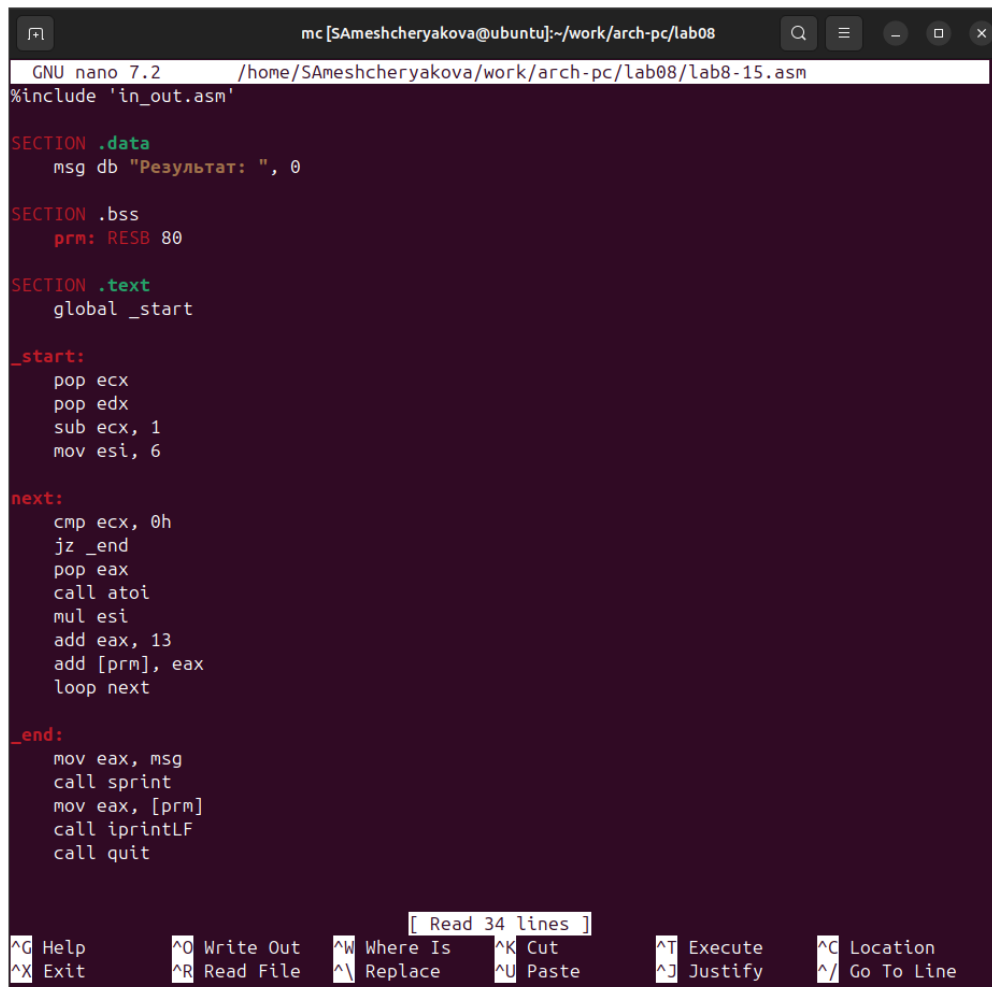
Рисунок 2.15: Запуск программы

2.3 Задание для самостоятельной работы

Вариант 15

1. Напишите программу, которая находит сумму значений функции $f(x)x = x_1, x_2, \dots, x_n$, т.е. программа должна выводить значение $f(x_1) + f(x_2) + \dots + f(x_n)$. Значения x_i передаются как аргументы. Вид функции $f(x)$ выбрать из таблицы 8.1 вариантов заданий в соответствии с вариантом, полученным при выполнении лабораторной работы № 7. Создайте исполняемый файл и проверьте его работу на нескольких наборах $x = x_1, x_2, \dots, x_n$.

Пишем программу, которая выведет сумму значений, получившихся после решения выражения $6x + 13$ (рис. 2.16).



```
mc [SAmeshcheryakova@ubuntu]:~/work/arch-pc/lab08
GNU nano 7.2 /home/SAmeshcheryakova/work/arch-pc/lab08/lab8-15.asm
#include 'in_out.asm'

SECTION .data
    msg db "Результат: ", 0

SECTION .bss
    prn: RESB 80

SECTION .text
    global _start

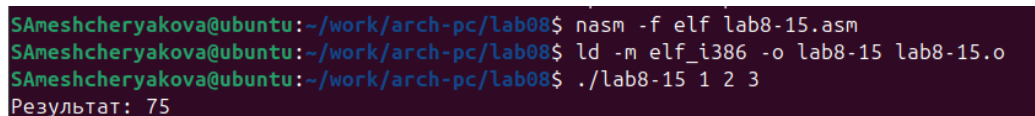
_start:
    pop ecx
    pop edx
    sub ecx, 1
    mov esi, 6

next:
    cmp ecx, 0h
    jz _end
    pop eax
    call atoi
    mul esi
    add eax, 13
    add [prn], eax
    loop next

_end:
    mov eax, msg
    call sprint
    mov eax, [prn]
    call iprintLF
    call quit
```

Рисунок 2.16: Текст программы для вычисления суммы значений

Создаём исполняемый файл и запускаем его, указав аргументы (рис. 2.18).



```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ nasm -f elf lab8-15.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-15 lab8-15.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ ./lab8-15 1 2 3
Результат: 75
```

Рисунок 2.17: Запуск программы (все верно)

Запустим файл при других значениях х (рис. 2.18).

```
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ nasm -f elf lab8-15.asm
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-15 lab8-15.o
SAmeshcheryakova@ubuntu:~/work/arch-pc/lab08$ ./lab8-15 1 2 3
Результат: 75
```

Рисунок 2.18: Запуск программы (все верно)

3 Выводы

Мы научились решать программы с использованием циклов и обработкой аргументов командной строки.